

实验 1：从软件测试到形式化验证

1. 实验主题

从熟悉的软件测试出发，使用 CBMC 检查二分查找中的整数中点计算，初步理解形式化工具如何检查一组输入、发现普通测试可能遗漏的整数溢出，并根据用户编写的断言检查功能要求。

2. 实验目标

- 为中点计算函数选择具体输入并执行测试，认识普通测试只能检查实际运行过的输入。
- 使用 C 有界模型检测器（C Bounded Model Checker, CBMC）检查所有满足输入条件的 `int` 取值，并从反例中找到触发有符号整数溢出（signed integer overflow）的输入。
- 理解验证入口程序（verification harness，简称 harness）中的非确定输入（nondeterministic input）、输入假设和用户断言。
- 区分“计算过程没有整数溢出”和“函数返回了正确中点”这两种不同结论。
- 把 CBMC 给出的反例转化为普通回归测试。

允许使用 LLM 查询 C 或 CBMC 语法，但测试选择、反例解释和验证结论必须由你根据实际运行结果完成。若使用了 LLM，应在实验报告中简要说明其用途。

3. 工具与参考资料

- [CBMC User Guide](#)
- [CBMC 命令说明](#)
- [CBMC Training: CBMC as unit testing](#)
- [CBMC Training: Integer overflow](#)

4. 最小环境检查（Smoke Test）

4.1 CBMC

保存以下文件为 `cbmc_smoke.c`：

```
int main(void) {
    int x = 1;
    __CPROVER_assert(x + 1 == 2, "one plus one");
    return 0;
}
```

运行：

```
cbmc cbmc_smoke.c --trace
```

环境正确时，应看到验证成功。

5. 实验任务

任务 1：测试整数中点计算函数

任务摘要： 为一个二分查找中常用的中点计算函数选择若干具体区间，执行普通测试并说明这些测试覆盖了哪些输入。

给定两个 `int` 类型的整数 `low` 和 `high`，本实验只考虑满足下列条件的输入：

```
0 <= low <= high
```

函数应返回闭区间 `[low, high]` 的较小中点，即按照数学整数计算 $\lfloor (low + high) / 2 \rfloor$ 。例如：

```
[10, 20] 的中点是 15  
[10, 21] 的较小中点是 15
```

保存以下文件为 `midpoint_test.c`：

```
#include <stdio.h>

int midpoint(int low, int high) {
    return (low + high) / 2;
}

int main(void) {
    int low;
    int high;

    if (scanf("%d%d", &low, &high) != 2) {
        return 1;
    }
    if (low < 0 || low > high) {
        puts("invalid input");
        return 0;
    }

    printf("%d\n", midpoint(low, high));
    return 0;
}
```

完成以下工作：

1. 自行选择并运行至少四组满足 `0 <= low <= high` 的输入。测试中应包含端点相等、端点不相等以及 `high - low` 为奇数和偶数的情况。根据数学上的整数中点确定正确结果，并记录程序的实际结果。
2. 列出全部已运行的 `(low, high)`，并另外写出至少三组满足输入条件但没有运行的 `(low, high)`。根据这两组输入，说明当前测试结果能够支持哪些具体输入上的判断，尚不能判断哪些输入的结果。

实验报告应包含： 每组测试的 `low`、`high`、根据数学定义得到的正确中点和程序实际结果，至少三组未运行的合法输入，以及根据这些输入得出的测试结论；测试运行的关键截图；以及你对“当前测试结果能够说明什么、不能说明什么”自行撰写的结论。

任务 2：使用 CBMC 检查整数溢出

任务摘要： 用非确定的 `low` 和 `high` 代替逐组测试，使用 CBMC 检查所有合法非负 `int` 区间中是否存在导致计算溢出的输入，并解释反例。

为了直接检查中点计算，下面的验证文件保留任务 1 中的 `midpoint`，并用非确定输入代替 `scanf`。保存为 `midpoint_cbmc.c`：

```
extern int nondet_int(void);

int midpoint(int low, int high) {
    return (low + high) / 2;
}

int main(void) {
    int low = nondet_int();
    int high = nondet_int();

    __CPROVER_assume(0 <= low && low <= high);

    int mid = midpoint(low, high);
    __CPROVER_assert(
        low <= mid && mid <= high,
        "midpoint is inside the interval");
    return 0;
}
```

运行：

```
cbmc midpoint_cbmc.c --signed-overflow-check --all-properties --trace
```

完成以下工作：

1. 在输出中找到 `low + high` 的有符号整数溢出性质，记录反例中的 `low` 和 `high`，并检查这两个数是否满足输入假设。
2. 使用数学整数计算反例中的 `low + high` 和较小中点，说明为什么数学上的中点仍可由 `int` 表示，而程序在计算它之前已经发生溢出。
3. 把反例中的 `low` 和 `high` 作为任务 1 程序的输入并实际运行，记录运行现象。说明普通运行的具体结果和 CBMC 判断“存在有符号整数溢出”分别告诉了了你什么。

实验报告应包含： CBMC 运行命令、失败的溢出性质、反例中的 `low` 和 `high`、数学计算与执行过程解释，以及使用反例进行普通测试的结果；能够看清失败性质、反例和普通运行现象的关键截图；以及你对 CBMC 如何检查合法输入范围自行撰写的结论。

任务 3：分别检查计算安全和功能要求

任务摘要：修正中点计算，再比较 CBMC 自动生成的整数溢出性质与自己编写的功能断言，理解两类检查回答的问题不同。

将任务 1 和任务 2 两个文件中的 `midpoint` 修正为：

```
int midpoint(int low, int high) {  
    return low + (high - low) / 2;  
}
```

修正后，将 `midpoint_cbmc.c` 中的 `main` 改为下面的验证入口。第一个断言检查结果是否仍在原区间中；第二个断言检查结果与 `low` 的距离是否等于区间长度的一半：

```
int main(void) {  
    int low = nondet_int();  
    int high = nondet_int();  
  
    __CPROVER_assume(0 <= low && low <= high);  
  
    int mid = midpoint(low, high);  
  
    __CPROVER_assert(  
        low <= mid && mid <= high,  
        "midpoint is inside the interval");  
    __CPROVER_assert(  
        mid - low == (high - low) / 2,  
        "midpoint has the expected offset");  
    return 0;  
}
```

完成以下工作：

1. 重新运行任务 1 的全部测试，记录修正后的实际结果。再对修正后的 `midpoint_cbmc.c` 运行任务 2 的 CBMC 命令，分别记录所有有符号整数溢出性质、第一个功能断言和第二个功能断言的结果。
2. 根据验证入口写明：`low` 和 `high` 可以取哪些值，输入假设排除了哪些调用，第一个断言检查什么，第二个断言检查什么。最后写出这次验证实际覆盖的输入条件和性质。
3. 暂时把 `midpoint` 改为下面这个不会计算 `low + high` 的版本，其它代码保持不变：

```
int midpoint(int low, int high) {  
    return low / 2 + high / 2;  
}
```

重新运行 CBMC，分别记录所有有符号整数溢出性质和两个功能断言的结果。从失败的功能断言中提取一组 `(low, high, mid)`，分别计算 `low / 2`、`high / 2`、二者之和以及正确的较小中点，指出实际结

果与正确结果的差异。完成后恢复正确实现。

实验报告应包含：修正前后的实现、普通测试结果、正确实现的溢出检查和功能断言结果、非确定输入与假设的含义，以及 `low / 2 + high / 2` 版本的各项性质结果、功能反例和整数除法计算；各阶段关键结果的截图；以及你对“没有整数溢出不等于功能正确”自行撰写的结论。

任务 4：比较软件测试与形式化验证

任务摘要：根据前面得到的实际结果，说明普通测试与 CBMC 分别检查了哪些输入，以及如何把形式化工具发现的问题加入日常测试。

- 1. 从任务 1 中选择一组通过的普通测试，再写出任务 2 的溢出反例。确认两组输入都满足 `0 <= low <= high`，并根据这两个实际结果说明为什么前一组测试通过不能排除后一组输入上的错误。
- 2. 把任务 2 的反例输入加入固定回归测试集，写出修正后的正确结果，并说明这条测试用于防止哪一种错误再次出现。
- 3. 根据任务 3 的最终结果写出验证结论，结论中应明确包含：适用的输入条件、使用的整数类型、已经检查的溢出性质和两个功能性质，以及至少两项本实验没有检查的输入或程序行为。

实验报告应包含：一组通过的普通测试与一组 CBMC 溢出反例的对照、由反例得到的回归测试与修正后结果，以及包含输入条件、整数类型、已检查性质和未检查内容的验证结论；支持这些判断的关键截图；以及你对软件测试与形式化验证如何配合自行撰写的结论。

6. 评分标准

本实验总分为 100 分。各项任务按照测试设计、实际运行结果、实验报告中的分析和截图综合评分。

任务	分值
任务 1：测试整数中点计算函数	25
任务 2：使用 CBMC 检查整数溢出	25
任务 3：分别检查计算安全和功能要求	35
任务 4：比较软件测试与形式化验证	15
总分	100